Hafta 7 Yerel Uygulama Yönergesi

Etkinlik Kaydı ve Eşzamansız Bildirim

Bulut Bilişim • Bilgisayar Mühendisliği Lisans Programı

1. Amaç ve Kapsam

Bu yönerge, Hafta7_Uygulama.zip içindeki hazır kodların çalıştırılmasını, değiştirilmesini ve deneylerle incelenmesini açıklar. Kayıt REST hizmeti, bildirim hizmeti ve RabbitMQ mesaj aracısı Docker Compose ile birlikte çalıştırılır. Kubernetes ve bulut hesabı gerekmez.

Öğrenci; REST isteklerini, hizmetlerin veri sorumluluklarını, kuyrukta beklemeyi, tekrar işleme korumasını ve kalıcı veriyi gözlemler. Gerçek e-posta veya kısa mesaj gönderilmez; bildirimin iş etkisi ayrı veri deposunda oluşturulan sonuç kaydıdır. Bulut FaaS dağıtımı bu yerel yönergenin kapsamına dahil değildir.

1.1. Ön Koşullar

- Docker Desktop çalışır durumda olmalıdır. Windows ortamında Linux konteynerleri kullanılmalıdır; Hafta 5 hazırlığı yeterlidir.
- Windows PowerShell 5.1 veya PowerShell 7 kullanılabilir. Komutları compose.yaml dosyasının bulunduğu klasörde çalıştırın.
- İlk kurulumda imajlar ve Pika bağımlılığı internetten indirilir. Ana bilgisayara Python, SQLite veya RabbitMQ kurmanız gerekmez.

1.2. Paket İçeriği

Dosya veya Klasör	Görevi
compose.yaml / Dockerfile	Üç hizmetin çalıştırılması ve Python hizmet imajının oluşturulması
kayit/	REST API, kontenjan kuralları ve outbox aktarım görevi
bildirim/	Mesaj tüketicisi, idempotent sonuç deposu ve sonuç API'si
ortak/	HTTP, veri bağlantısı ve mesajlaşma yardımcıları
ornekler/	istekler.http ve deney.ps1 örnekleri
scripts/ / tests/	Uçtan uca kontrol aracı, geçersiz mesaj deneyi ve 17 test
README.md / API_SOZLESMESI.md / DOGRULAMA.md	Başlangıç açıklamaları, arayüz kuralları ve doğrulama kapsamı

Kodları sıfırdan yazmanız beklenmez. Kaynaklar Python ile hazırlanmıştır; çalışma boyunca küçük doğrulama değişikliklerine ve sistem davranışına odaklanın.

2. Sistemi Başlatma

2.1. Dosyaları Açma ve Klasörü Seçme

ZIP dosyasını açın. İçindeki Hafta7_Uygulama klasörüne girin ve burada PowerShell açın. Klasör içinde compose.yaml, Dockerfile ve hizmet klasörleri görünmelidir. ZIP'i açmadan içinden komut çalıştırmayın.

```
Get-Location
Get-ChildItem
```

2.2. Yapılandırmayı Kontrol Etme ve Başlatma

```
docker compose config
docker compose up --build -d
docker compose ps
```

İlk başlatmada indirme ve imaj oluşturma zaman alabilir. Mesaj aracısının healthcheck kontrolü başarılı olduktan sonra uygulama hizmetleri başlatılır. ps çıktısında mesaj-aracisi, kayit ve bildirim hizmetlerinin çalıştığını kontrol edin.

```
docker compose logs --tail 50 kayit bildirim
docker compose logs --tail 50 mesaj-aracisi
```

Kayıt logunda mesaj aracısına bağlantı, bildirim logunda tüketicinin hazır olduğu bilgisi aranır. Sadece HTTP sağlık kontrolünün başarılı olması, mesajın uçtan uca işlendiğini kanıtlamaz.

2.3. Erişim Adresleri

Adres	Kullanım
http://localhost:8080/etkinlikler	Etkinlik listesi
http://localhost:8081/sonuclar	Tamamlanan bildirim sonuçları
http://localhost:15672	RabbitMQ yönetim arayüzü
http://localhost:8080/health **http://localhost:8081/health**	HTTP hizmetlerinin sağlık kontrolü

RabbitMQ arayüzü için kullanıcı ders, parola hafta7-yerel değeridir. Bu değerler yalnız yerel eğitim kurulumu içindir. Host portları 127.0.0.1'e bağlıdır; RabbitMQ'nun mesajlaşma portu ana bilgisayara açılmaz.

Tarayıcı GET sorguları için kullanılabilir. POST deneyleri PowerShell, isteğe bağlı Postman veya VS Code REST Client ile yapılır. Windows PowerShell'de curl adı bir takma ad olabilir; curl kullanılacaksa curl.exe yazın.

3. Etkinlikleri Görüntüleme ve Kayıt Oluşturma

3.1. Başlangıç Verisini İnceleme

```
Invoke-RestMethod -Uri http://localhost:8080/etkinlikler
```

Etkinlik Kimliği	Başlangıç Durumu
42	Bulut Bilişim Atölyesi • 5 kişilik kontenjan • kayıt açık
43	Mikroservis Semineri • 2 kişilik kontenjan • kayıt açık
44	Kayıtları Kapanmış Etkinlik • kayıt kapalı

Veriler yalnız ilk açılışta eklenir. Konteyneri yeniden başlatmak mevcut kayıtları sıfırlamaz. Deneyler kontenjan tüketir; tekrar çalışmada daha önce kullanılmamış katılımcı kimliklerini seçin.

3.2. İlk Kaydı Oluşturma

```
$body = @{etkinlikId=42; katilimciId=108} | ConvertTo-Json
$r = Invoke-RestMethod -Uri http://localhost:8080/kayitlar `
  -Method Post -ContentType 'application/json' -Body $body
$r | ConvertTo-Json
```

Geçerli istek 201 Created yanıtı üretir. Yanıtta kayitId, durum, bildirimIsId ve aktarimDurumu alanlarını inceleyin. $r değişkenini aynı PowerShell oturumunda sonraki adımlarda kullanacağız. Location başlığı kaydın adresini gösterir; Invoke-RestMethod gövdeyi doğrudan nesneye dönüştürür.

3.3. Kaydı ve Bildirimi Ayrı Sorgulama

```
Invoke-RestMethod -Uri ("http://localhost:8080/kayitlar/" + $r.kayitId)
Invoke-RestMethod -Uri http://localhost:8081/sonuclar
```

Kayıt durumu onaylandi iken bildirim henüz tamamlanmamış olabilir. Bildirimde yaklaşık bir saniyelik öğretim amaçlı işlem gecikmesi vardır; ilk sorguda sonuç görünmezse yeniden sorgulayın. Sonuç listesindeki isId ile $r.bildirimIsId aynı işi tanımlar.

Yeni PowerShell oturumu açılırsa $r kaybolur. Gerçek kayıt kimliğinizi kullanarak tekrar yükleyin:

```
$r = Invoke-RestMethod -Uri http://localhost:8080/kayitlar/1
```

Buradaki 1 örnektir; her öğrenci yanıtta aldığı kayitId değerini kullanmalıdır. Eksik bildirim sonucu için /sonuclar/{isId} sorgusunun 404 vermesi, kaydın başarısız olduğu anlamına gelmez.

4. Hatalı İstekleri ve İş Kurallarını Sınama

Aşağıdaki POST isteklerini ayrı ayrı çalıştırın. Hata yanıtı PowerShell'de hata mesajı olarak görünür; bu deneyde beklenen davranıştır. Komutu düzeltip başarılı yapmaya çalışmadan önce HTTP kodunu ve API hata kodunu kaydedin.

4.1. Aynı Kaydı Tekrar İsteme

```
Invoke-RestMethod -Uri http://localhost:8080/kayitlar `
  -Method Post -ContentType 'application/json' `
  -Body '{"etkinlikId":42,"katilimciId":108}'
```

Beklenen: 409 Conflict ve MEVCUT_KAYIT. Önceki kayıt korunur, ikinci kayıt ve yeni bildirim işi oluşmaz. İlk deneyde başka katılımcı kullandıysanız burada da aynı kimliği kullanın.

4.2. Eksik Alan Gönderme

```
Invoke-RestMethod -Uri http://localhost:8080/kayitlar `
  -Method Post -ContentType 'application/json' -Body '{"etkinlikId":42}'
```

Beklenen: 400 Bad Request ve GECERSIZ_ALAN. katilimciId zorunludur. Kimliklerin pozitif tam sayı olması gerekir; metin biçimindeki "108", boolean, sıfır veya negatif değerler kabul edilmez.

4.3. Bulunamayan ve Kapalı Etkinlik

```
Invoke-RestMethod -Uri http://localhost:8080/kayitlar `
  -Method Post -ContentType 'application/json' `
  -Body '{"etkinlikId":999,"katilimciId":108}'

Invoke-RestMethod -Uri http://localhost:8080/kayitlar `
  -Method Post -ContentType 'application/json' `
  -Body '{"etkinlikId":44,"katilimciId":108}'
```

Beklenen: ilk istekte 404 ve ETKINLIK_YOK; ikinci istekte 409 ve KAYIT_KAPALI. Kontenjan dolu açık etkinlik ise 409 ve KONTENJAN_DOLU üretir. Kontenjan deneyini, gerekli kayıtları oluşturduktan sonra yapın; temel deneyler için boş yer bırakın.

Yanıt gövdesini görmek zorlaşırsa ornekler/istekler.http dosyasını REST Client veya Postman ile kullanın. İçerik türü application/json olmalıdır; başka türdeki kayıt isteği 415 yanıtı alır.

5. Bildirim Hizmetini Durdurma ve Yeniden Başlatma

5.1. Tüketiciyi Durdurma

```
docker compose stop bildirim
docker compose ps
```

Yalnız bildirim hizmetini durdurun. Kayıt hizmeti ve mesaj aracısı çalışmaya devam etmelidir. Bu deney sırasında sonuç API'si de bildirim konteynerinde bulunduğu için 8081 adresine erişilemez; bu beklenen durumdur.

5.2. Yeni Bir Kayıt Oluşturma

```
$body2 = @{etkinlikId=42; katilimciId=109} | ConvertTo-Json
$r2 = Invoke-RestMethod -Uri http://localhost:8080/kayitlar `
  -Method Post -ContentType 'application/json' -Body $body2
$r2 | ConvertTo-Json
Invoke-RestMethod -Uri http://localhost:8080/outbox
```

Beklenen: kayıt tamamlanır. 109 daha önce kullanıldıysa başka bir kimlik seçin. Outbox çıktısındaki iletilmeZamani, mesajın aracıya aktarıldığını belirtir; bildirimin tamamlandığını belirtmez. Kayıt yanıtındaki aktarimDurumu ilk anda bekliyor olabilir; aktarım görevi daha sonra günceller.

5.3. Kuyrukta Bekleyen İşi Görme

http://localhost:15672 adresinde oturum açın. Queues and Streams bölümünde hafta7.bildirim kuyruğunu açın. Ready sayacı bekleyen mesajları gösterir. Unacked sayacı alınmış fakat henüz tamamlanma onayı verilmemiş mesajlarla ilişkilidir. Görüntü yenilenmesi kısa süre gecikebilir.

Bu aşamada yönetim arayüzündeki Get messages işlemini kullanmayın; mesaj alma veya silme ayarları deney sonucunu değiştirebilir. Kuyruk sayaçlarını incelemek yeterlidir.

5.4. Tüketiciyi Yeniden Başlatma

```
docker compose start bildirim
docker compose logs --tail 50 bildirim
Invoke-RestMethod -Uri http://localhost:8081/sonuclar
```

Başlangıçtan ve işleme süresinden sonra $r2.bildirimIsId değerine ait sonuç görünmelidir. Ready sayısı azalır; başarılı iş kalıcı olarak saklandıktan sonra mesaj onaylanır. Aynı kayıt için kullanıcıya verilen kayıt yanıtı değişmez.

Raporda bildirim hizmeti kapalıyken kayıt neden tamamlandı ve yeniden başladıktan sonra iş nasıl sürdü sorularını yanıtlayın.

6. Aynı Bildirim İşini Tekrar Gönderme

6.1. Mevcut Sonucu Kaydetme

Bildirim hizmeti çalışmalı ve ilk kaydın bildirimi tamamlanmış olmalıdır. Aşağıdaki sorguda tek sonuç elde edildiğini kontrol edin. Boş sonuç varsa kısa süre sonra yeniden sorgulayın.

```
$once = @(Invoke-RestMethod -Uri http://localhost:8081/sonuclar |
  Where-Object { $_.isId -eq $r.bildirimIsId })
$once | ConvertTo-Json
```

tamamlanmaZamani ve teslimSayisi değerlerini kaydedin. Tekrar deneyi yeni bir kayıt oluşturmaz; mevcut kaydın aynı bildirim işi yeniden iletilir.

6.2. Aynı İş Kimliğiyle Yeniden İletme

```
Invoke-RestMethod -Uri `
  ("http://localhost:8080/kayitlar/" + $r.kayitId + "/bildirimi-tekrarla") `
  -Method Post
```

Beklenen: 202 ve tekrar_aktarim_bekliyor. Yeni bir outbox satırı oluşturulur, fakat isId korunur. Sonuç hemen hazır olmak zorunda değildir.

6.3. Sonucu Karşılaştırma

```
$sonra = @(Invoke-RestMethod -Uri http://localhost:8081/sonuclar |
  Where-Object { $_.isId -eq $r.bildirimIsId })
$sonra | ConvertTo-Json
docker compose logs --tail 30 bildirim
```

İşlendikten sonra aynı isId için sonuç sayısı 1 kalmalıdır. teslimSayisi artar; tamamlanmaZamani ve bildirim sonucu korunur. Logda TEKRAR_TANINDI görülür. Hemen artış yoksa aktarım ve tüketim tamamlanana kadar sorguyu yeniden çalıştırın.

Bu sayaç başarılı yeni bildirim sayısı değildir; sonuç deposunun aynı geçerli işi kaç kez işlediğini gösterir. Yeni isId üretmek farklı bir işi tanımlar ve bu deneyin yerine geçmez.

6.4. Korumanın Sınırı

Örnekte iş etkisi SQLite sonuç satırıdır. isId birincil anahtarı ve transaction, aynı işi eşzamanlı alan işçilerin tek sonuç oluşturmasını sağlar. Gerçek e-posta ya da ödeme gibi dış yan etkiler için yalnız bu yerel kayıt yeterli olmayabilir; dış hizmetin tekrar önleme desteği de gerekir.

7. Kalıcılık ve Küçük Kod Değişikliği

7.1. Konteynerleri Yeniden Oluşturma

KayitId, bildirimIsId ve sonucu kaydedin. Ardından kalıcı hacimleri koruyarak sistemi kapatıp tekrar başlatın:

```
docker compose down
docker compose up -d
docker compose ps
```

Hizmetler hazır olduktan sonra önceki kayıt ve sonucu sorgulayın. Aynı PowerShell oturumundaysanız $r kullanılabilir; yeni oturumda gerçek kimlikle kaydı tekrar yükleyin.

```
Invoke-RestMethod -Uri ("http://localhost:8080/kayitlar/" + $r.kayitId)
Invoke-RestMethod -Uri http://localhost:8081/sonuclar
```

Beklenen: kayıt ve bildirim sonucu korunur. kayit-verisi, bildirim-verisi ve mesaj-verisi ayrı named volume alanlarıdır. Bu deneyde down -v kullanmayın; -v kalıcı hacimleri siler.

7.2. Doğrulama Kuralını Değiştirme

kayit/store.py dosyasında KATILIMCI_ID_UST_SINIRI sabitini bulun. Başlangıç değeri 999999'dur. Örnek değişiklik olarak 1000 yapın ve dosyayı kaydedin.

```
KATILIMCI_ID_UST_SINIRI = 1000
```

Kod değişikliğinin çalışan imaja geçmesi için kayıt hizmetini yeniden oluşturun:

```
docker compose up --build -d kayit
```

katilimciId=1001 değerli istek artık 400 ve KATILIMCI_SINIRI yanıtı üretmelidir. Sınır içinde ve daha önce kullanılmamış bir değerle geçerli istek deneyin; örneğin boş kontenjan varsa etkinlik 43, katılımcı 500. Önceki kayıtlar bu kod değişikliğiyle silinmez.

Rapora değişiklik öncesi/sonrası beklentinizi, değiştirilen satırı ve sonucu ekleyin. Değişiklik testini tamamladıktan sonra sabiti 999999 yapıp aynı build komutunu tekrar çalıştırın. Otomatik kontrol aracı büyük katılımcı kimliği ürettiği için sınırın geri alınması önemlidir.

7.3. İşin Aracıya Aktarılamadığı Durum

Mesaj aracısına geçici erişim kesilirse kayıt API'si çalışmaya devam edebilir; iletilecek işler outbox içinde kalır ve aktarım görevi bağlantıyı yeniden dener. /outbox içindeki denemeSayisi ve sonHata bu durumu açıklamaya yardımcı olur. Bildirim tüketicisini durdurmakla mesaj aracısını durdurmak aynı deney değildir.

8. İsteğe Bağlı İleri Deneyler

Bu deneyler temel çalışmayı tamamladıktan sonra uygulanır. Gerçek dış mesaj gönderimi yoktur. Önce yeni kayıt için etkinlik 42'de boş kontenjan bulunduğunu kontrol edin.

8.1. Sonuçtan Sonra Onay Kaybı

```
Invoke-RestMethod -Uri `
  http://localhost:8081/deneyler/onay-oncesi-kapanma -Method Post
```

Bu istek bir defalık kapanma deneyini hazırlar. Sonraki yeni bildirim işinde sonuç kalıcı olarak yazılır; ardından mesaj onaylanmadan süreç kapanır. Deney işareti sonuçla birlikte temizlenir; aynı işte sonsuz kapanma döngüsü oluşmaz.

Daha önce kullanılmamış katılımcı kimliğiyle kayıt oluşturun:

```
$body3 = @{etkinlikId=42; katilimciId=110} | ConvertTo-Json
$r3 = Invoke-RestMethod -Uri http://localhost:8080/kayitlar `
  -Method Post -ContentType 'application/json' -Body $body3
docker compose logs --tail 70 bildirim
```

Compose, restart: unless-stopped ayarıyla çöken konteyneri yeniden başlatır. RabbitMQ onaysız mesajı yeniden teslim edebilir. Loglarda SONUC_OLUSTURULDU, deney kapanma mesajı ve TEKRAR_TANINDI izlenir. Sonuç tek satır olmalı, teslimSayisi artmalıdır. Kapanma sırasında 8081 sorgusu kısa süre başarısız olabilir; hazır olunca yeniden deneyin.

8.2. Geçersiz Mesajı Ayırma

```
docker compose exec kayit python scripts/gecersiz_mesaj.py
```

Araç, zorunlu alanları eksik geçersiz mesaj gönderir. Bildirim logunda geçersiz iş bilgisi görülmeli; RabbitMQ arayüzündeki hafta7.basarisiz kuyruğunda mesaj bulunmalıdır. Bu kuyruk DLQ işlevi görür.

Geçersiz mesajlar normal kuyruğa sonsuza kadar geri gönderilmez. Depo kaynaklı geçici hatalar için en fazla 3 uygulama yeniden denemesi yapılır; sınıra ulaşıldığında mesaj ayrılır. Bağlantı kopmasına bağlı broker yeniden teslimleri bu uygulama sayacıyla aynı şey değildir. DLQ'ya ayrılmak, hatanın giderildiği anlamına gelmez.

9. Otomatik Kontroller

9.1. Gerçek Compose Akışını Doğrulama

Üç hizmet çalışırken aşağıdaki komutu kullanın. KATILIMCI_ID_UST_SINIRI değeri başlangıçtaki 999999 olmalıdır. Açık etkinliklerden en az birinde boş yer gerekir.

```
docker compose exec kayit python scripts/smoke_test.py
```

Araç yeni bir kayıt oluşturur, gerçek RabbitMQ üzerinden bildirimin tamamlanmasını bekler, tekrar kaydın reddini kontrol eder ve aynı bildirim işini yeniden gönderir. Tek sonuç korunuyorsa BASARILI çıktısı üretir. Kontrol başarısızsa sıfırdan farklı çıkış kodu verir; yaklaşık 45 saniyelik sonuç bekleme sınırı vardır.

Bu kontrol yeni bir kontenjan tüketir. Veriyi otomatik silmez; kontenjan yoksa açıklama ile durur. 17 birim/HTTP testi ile aynı test değildir; gerçek mesajlaşma bağlantısını ayrıca sınar.

9.2. Birim ve HTTP Sözleşme Testleri

Ana bilgisayarda Python 3.12 varsa paket klasöründe:

```
python -m unittest discover -s tests -v
```

Ana bilgisayara Python kurmadan Docker içinde çalıştırmak için aynı proje klasöründe PowerShell kullanın:

```
docker compose run --rm --no-deps -v "${PWD}:/test" -w /test `
  kayit python -m unittest discover -s tests -v
```

Bu testler geçici veri depoları kullanır; ders uygulamasının kalıcı kayıtlarını değiştirmez. RabbitMQ gerektirmez. Beklenen çıktı 17 test ve OK bilgisidir.

9.3. Doğrulamanın Kapsamı

Paket hazırlanırken 17 test geçirilmiştir: eşzamanlı kontenjan ve tekrar kayıt, outbox yazma hatasında geri alma, aynı işin eşzamanlı tekrarları, kalıcılık ve HTTP yanıtları kontrol edilmiştir. Python kaynakları ve Compose YAML yapısı da incelenmiştir.

Hazırlama ortamında Docker ve RabbitMQ bulunmadığı için imaj oluşturma ve gerçek Compose akışı burada çalıştırılarak doğrulanmamıştır. Gerçek mesaj akışı için 9.1'deki kontrolü kendi Docker Desktop ortamınızda uygulayın. Paketteki DOGRULAMA.md bu ayrımı açıklar.

9.4. Sağlık Kontrolünü Yorumlama

/health yanıtı HTTP hizmetinin çalıştığını belirtir. Kayıt API'sinin sağlıklı olması outbox'ın boş olduğunu, bildirim API'sinin sağlıklı olması tüketicinin broker bağlantısının hazır olduğunu göstermez. Uçtan uca doğrulamada kayıt, aktarım ve sonuç birlikte kontrol edilir.

10. Rapor ve Teslim

Kısa raporunuzu PDF olarak hazırlayın. Yalnızca konteynerlerin çalıştığını gösteren görüntüler yeterli değildir; her kanıtın neyi gösterdiğini bir veya iki cümleyle açıklayın.

10.1. Rapor İçeriği

Kısım	Gerekli İçerik
Bileşenler	Üç hizmetin görevi, ayrı veri depoları ve temel işlem akışı
REST deneyleri	Geçerli kayıt, eksik alan, mevcut kayıt ve kapalı/bulunamayan etkinlik yanıtları
Kuyruk deneyi	Bildirim kapalıyken kayıt sonucu, kuyrukta bekleyen iş ve yeniden başlatma sonrası sonuç
Tekrar işleme	Aynı isId için önce/sonra sonucu; tek satır, artan teslimSayisi
Kalıcılık	Hacimleri koruyarak yeniden oluşturma sonrasında eski kayda erişim
Kod değişikliği	Değiştirilen satır, gerekçe, değişiklik sonrası test ve beklenen davranış
Kısa değerlendirme	Sonucu iletişim ayrımı, idempotentlik ve veri sahipliğiyle ilişkilendirme

İsteğe bağlı onay kaybı veya DLQ deneyi yaptıysanız ayrı alt başlıkta sunun. Otomatik kontrol çıktısı destekleyici kanıttır; açıklama yerine geçmez.

10.2. Teslim Dosyaları

Raporu OgrenciNo_Hafta7_Rapor.pdf adıyla kaydedin. Değiştirdiğiniz kaynak dosyalarını paket içindeki klasör yolunu koruyarak OgrenciNo_Hafta7_Degisiklikler.zip içine koyun; bu çalışmada genellikle kayit/store.py yeterlidir. Raporda sınırı geri aldıysanız denenen değişiklik ve test sonucu açıkça görünmelidir.

Hazır paketin tümünü, konteyner imajlarını, veritabanı dosyalarını veya kişisel bağlantı bilgilerini yeniden teslim etmeyin. PDF ile değişiklik ZIP'ini dersin öğretim elemanının belirttiği teslim alanına yükleyin. Son tarih ve puanlandırma, ders duyurusunda belirtilen kurallara göre uygulanır.

10.3. Değerlendirme Ölçütü

Arayüzü doğru kullanma, küçük değişikliği doğrulama, deneyleri kanıtlarla açıklama ve sistem davranışını kavramlarla ilişkilendirme esas alınır. Kullanılan ürün sayısı veya yalnızca hizmetleri başlatabilmek başarı ölçütü değildir. Bulut genişletmesi yerel çalışmanın yerine geçmez ve ayrı değerlendirilir.

11. Sorun Giderme ve Sistemi Kapatma

Belirti	Kontrol veya Çözüm
compose.yaml bulunamadı	Get-Location ve Get-ChildItem ile doğru klasörde olduğunuzu kontrol edin.
Port zaten kullanılıyor	compose.yaml içindeki host portunu değiştirin; localhost isteklerini de uyarlayın. Konteyner portunu koruyun.
Mesaj aracısı hazır değil	docker compose ps ve logs mesaj-aracisi ile healthcheck ve başlangıç kayıtlarını inceleyin.
Kayıt 409 veriyor	Hata kodunu okuyun: katılımcı zaten kayıtlı, etkinlik kapalı veya kontenjan dolu olabilir.
Bildirim sonucu yok	/outbox, bildirim logları ve hafta7.bildirim kuyruk sayaçlarını birlikte inceleyin.
Kod değişikliği görünmüyor	Dosyayı kaydedip docker compose up --build -d kayit çalıştırın.
8081 adresi erişilemiyor	Bildirim konteyneri durmuş veya kapanma deneyi sırasında yeniden başlıyor olabilir.
Veri deposu hatası	Kalıcı hacim izinlerini ve hizmet loglarını inceleyin. SQLite bu tek makine örneği içindir.

11.1. Port Değişikliği Örneği

8080 doluysa kayit hizmetinin ports satırında yalnız host tarafını örneğin 18080 yapın:

```
- "127.0.0.1:18080:8080"
```

Ardından docker compose up -d ile uygulayın. Hosttaki kayıt istekleri artık localhost:18080 kullanır. Konteyner içindeki healthcheck ve smoke_test.py hâlâ kayit:8080 adresine erişir. Aynı mantık 8081 ve 15672 için de geçerlidir.

11.2. Veriyi Koruyarak Kapatma

```
docker compose down
```

Bu komut proje konteynerlerini ve ağını kaldırır; named volume verileri korunur. Daha sonra docker compose up -d ile yeniden başlatabilirsiniz.

11.3. Tamamen Sıfırlama

Yalnız rapor kanıtlarınızı aldıktan ve deney verisine ihtiyaç kalmadıktan sonra kullanın. Aşağıdaki komut kayıtları, sonuçları ve kuyruk verisini kalıcı olarak siler:

```
docker compose down -v
```

Bu işlem kalıcılık deneyi veya otomatik kontrollerin parçası değildir. Sıfırladıktan sonra başlatma komutuyla başlangıç etkinlikleri yeniden oluşturulur.

12. Mimari Açıklamalar ve Başvuru

12.1. Kodda İzlenecek Akış

kayit/store.py içindeki create işlemi, kontenjan kontrolünü ve kayıt oluşturmayı BEGIN IMMEDIATE ile aynı transaction içinde korur. Kayıt ile outbox işi birlikte saklanır. kayit/app.py içindeki relay görevi bekleyen işleri iletir; yayıncı onayından sonra satırı iletildi olarak işaretler. Onay ile yerel işaretleme arasındaki kapanmada tekrar yayın mümkündür.

bildirim/store.py içindeki process, isId başına bir sonucu transaction içinde saklar. Tekrar içerik aynıysa mevcut sonucu korur ve teslim sayısını artırır. Aynı iş kimliği farklı içerikle gelirse geçersiz kabul edilir. bildirim/app.py sonucu commit ettikten sonra mesaj onayı verir.

12.2. Verilerin ve Adreslerin Anlamı

Alan veya Adres	Anlam
kayitId	Kalıcı katılımcı kaydının kimliği
bildirimIsId / isId	Aynı mantıksal bildirim işini tanıyan kimlik
aktarimDurumu / outbox	İşin mesaj aracısına aktarımı; tüketicide tamamlanma değildir
teslimSayisi	Aynı geçerli iş için sonuç deposunun işleme sayısı
tamamlanmaZamani	İlk başarılı bildirim sonuç kaydının zamanı
/sonuclar/{isId}	Sonuç henüz yoksa 404; temel kaydın başarısızlığı değildir

12.3. Öğretim Amaçlı Sınırlar

Uygulama tek RabbitMQ düğümü, iki ayrı SQLite dosyası ve localhost'a açık deney uç noktaları kullanır. Kimlik doğrulama ve üretim ölçekli yüksek erişilebilirlik içermez. Ortak yardımcı kod kullanılması hizmetlerin iş verisini paylaştığı anlamına gelmez. İki hizmet birbirinin SQLite dosyasını açmaz.

Tek düğüm ve yerel hacimlerin korunması, çoklu bölge veya donanım kaybına karşı güvence değildir. Mesaj kalıcılığı, kuyruk kalıcılığı, yayıncı onayı ve tüketici onayı ayrı mekanizmalardır. Gerçek dış yan etkiler ve buluta taşıma, ek tasarım gerektirir. Deney uç noktalarını internete yayımlamayın.

12.4. Teknik Kaynaklar

RabbitMQ — Work Queues: https://www.rabbitmq.com/tutorials/tutorial-two-python

Pika — Python istemci belgeleri: https://pika.readthedocs.io/

Docker — Compose başlangıç sırası: https://docs.docker.com/compose/how-tos/startup-order/

Paketle birlikte API_SOZLESMESI.md bütün uç noktaların, README.md hızlı başlangıcın, DOGRULAMA.md ise gerçekleştirilen kontrollerin referansıdır. Bu yönerge aynı kod paketi ve aynı varsayımlar üzerinden hazırlanmıştır.